网络流竞赛板子

本板子默认 C++17、点编号从 `1` 到 `n`。容量、流量、费用和答案统一使用 `ll`；结构体名与局部变量尽量短，适合赛场直接复制。

除特别说明外，所有 `add(u,v,c)` 都表示加入一条容量为 `c` 的**有向边** `u -> v`。不要把题目中的边权误当成容量。

0. 公共约定与算法选择

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll INF = (1LL << 62);
```

`INF` 保留了少量运算余量，但所有合法流量、费用总和及 `流量 * 单价` 仍须能装进 `ll`。

问题	首选算法	说明
普通最大流、最小割	Dinic	短、稳、能方便查询残量网络
稠密图或大规模最大流	HLPP	常数优秀，但代码更长且是一次性模板
二分图最大匹配	Hopcroft-Karp	纯匹配通常比网络流更快
带上下界可行流/最大流/最小流	下界消除 + Dinic	用超级源汇检查可行性
最小费用最大流/定流量最小费用	势能 Dijkstra	支持负费用边，无负费用环
带上下界最小费用可行流	费用边预饱和 + 势能 Dijkstra	支持正负费用与点供需
无向图任意两点最小割	Gomory-Hu Tree	`n-1` 次最大流后树上查询
无向图全局最小割	Stoer-Wagner	只求全局答案时通常更直接
有向图全局最小割	固定根双向枚举	`2(n-1)` 次最大流，写法稳定

算法取舍：Edmonds-Karp 主要用于教学，性能明显弱于 Dinic；ISAP/SAP 与 Dinic、HLPP 能解决的问题完全重合；ZKW 费用流没有增加题型覆盖，最坏性能也不比势能 Dijkstra 稳定。因此不重复收录。Hao-Orlin、有向割树、cost-scaling 等超长专用算法只在极端专项题有优势，不适合作为常规赛场抄写板子。

最大流等于最小割容量。Dinic 的最坏复杂度是 `O(n^2m)`；单位容量网络上可达到 `O(m sqrt(m))`，二分图匹配型网络常写作 `O(m sqrt(n))`。实际表现通常远好于最坏界。

1. Dinic：最大流主模板

适合绝大多数最大流、最小割和建模题。使用链式前向星保存边，避免 `vector<vector<Edge>>` 的额外分配；当前弧优化和分层剪枝可显著降低常数。

- `Dinic f(n,m)`：建立 `1..n` 的网络，`m` 是预计加入的原边数，仅用于预留空间，可省略。
- `add(u,v,c,rc)`：加入 `u -> v`，正向初始容量 `c`、反向初始容量 `rc`，返回正向边编号。普通有向边令 `rc=0`。
- `maxflow(s,t,lim)`：在当前残量网络上继续增广，最多再送 `lim` 流量，返回本次新增流量。
- `flow(id)`：查询编号 `id` 的边当前净流量；`cap(id)`、`rem(id)` 分别为初始容量和剩余容量。
- `cut(s)`：最大流结束后，返回残量网络中从 `s` 可达的点集；可达与不可达两侧构成一个最小割。
- `rcut(t)`：最大流结束后，返回残量网络中能够到达 `t` 的点集，用于分析全部最小割。
- `reset()`：恢复所有边的初始容量，可在同一张图上重新跑最大流。

```
struct Dinic {
    struct E { int v, nx; ll c, oc; };
    int n;
    vector<int> h, lv, cur;
    vector<E> e;

    Dinic(int n = 0, int m = 0) { init(n, m); }
    void init(int n_, int m = 0) { // n_: 点数; m: 原边数预估
        n = n_;
        h.assign(n + 1, -1);
        lv.resize(n + 1), cur.resize(n + 1);
        e.clear(), e.reserve(2 * m);
    }
    int add(int u, int v, ll c, ll rc = 0) { // u->v 容量c; rc: 初始反向容量
        int id = (int)e.size();
        e.push_back({v, h[u], c, c}), h[u] = id;
        e.push_back({u, h[v], rc, rc}), h[v] = id ^ 1;
        return id;
    }
    bool bfs(int s, int t) {
        fill(lv.begin(), lv.end(), -1);
        queue<int> q;
        lv[s] = 0, q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int i = h[u]; i != -1; i = e[i].nx) {
                int v = e[i].v;
                if (e[i].c > 0 && lv[v] == -1)
                    lv[v] = lv[u] + 1, q.push(v);
            }
        }
        return lv[t] != -1;
    }
```

```
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        ll r = f;
        for (int &i = cur[u]; i != -1 && r; i = e[i].nx) {
            int v = e[i].v;
            if (e[i].c > 0 && lv[v] == lv[u] + 1) {
                ll z = dfs(v, t, min(r, e[i].c));
                e[i].c -= z, e[i ^ 1].c += z, r -= z;
            }
        }
        if (r == f) lv[u] = -1;
        return f - r;
    }
    ll maxflow(int s, int t, ll lim = INF) { // s,t: 源汇; lim: 最多新增流量
        if (s == t || lim <= 0) return 0;
        ll ans = 0;
        while (ans < lim && bfs(s, t)) {
            cur = h;
            ans += dfs(s, t, lim - ans);
        }
        return ans;
    }
    vector<char> cut(int s) const { // s: 残量网络搜索起点
        vector<char> vis(n + 1);
        queue<int> q;
        vis[s] = 1, q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int i = h[u]; i != -1; i = e[i].nx) {
                int v = e[i].v;
                if (e[i].c > 0 && !vis[v]) vis[v] = 1, q.push(v);
            }
        }
        return vis;
    }
    vector<char> rcut(int t) const { // t: 反向残量搜索起点
        vector<char> vis(n + 1);
        queue<int> q;
        vis[t] = 1, q.push(t);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int i = h[u]; i != -1; i = e[i].nx) {
                int v = e[i].v;
                if (e[i ^ 1].c > 0 && !vis[v]) vis[v] = 1, q.push(v);
            }
        }
        return vis;
    }
    ll flow(int id) const { return e[id].oc - e[id].c; } // id: add返回值
    ll cap(int id) const { return e[id].oc; }            // id: add返回值
    ll rem(int id) const { return e[id].c; }             // id: add返回值
    void reset() { for (auto &x : e) x.c = x.oc; }
```

```
    void off(int id) { // id: 要彻底移除的一对残量边
        e[id].c = e[id].oc = 0;
        e[id ^ 1].c = e[id ^ 1].oc = 0;
    }
};
```

调用示例：

```
int n, m, s, t;
cin >> n >> m >> s >> t;
Dinic f(n, m);
for (int i = 0, u, v; i < m; i++) {
    ll c;
    cin >> u >> v >> c;
    f.add(u, v, c); // u,v: 端点; c: 容量
}
cout << f.maxflow(s, t) << '\n'; // s: 源点; t: 汇点
```

如果题目给一条容量为 `c` 的无向边，使用 `f.add(u,v,c,c)`。它允许净流量在 `[-c,c]` 内变化，求最大流/最小割时正是无向容量边的含义。

递归 `dfs` 的深度最多为点数。若图可能出现数十万层的长链，应增大栈空间或改写成非递归增广；一般 XCPC 数据可直接使用。

1.1 查询边流量与最小割边

加入边时保存 `add` 返回的编号，即可在跑完后查询每条原边的流量。

```
struct OE { int u, v, id; };
vector<OE> a;
int id = f.add(u, v, c); // u,v,c: 原边信息
a.push_back({u, v, id});

ll ans = f.maxflow(s, t);
auto vis = f.cut(s);
for (auto [u, v, id] : a) {
    ll x = f.flow(id);                // x: 该有向边净流量
    if (vis[u] && !vis[v] && f.cap(id) > 0) {
        // 该边属于当前得到的一组最小割边
    }
}
```

必须先把最大流跑满再调用 `cut(s)`。若使用了 `lim` 且提前停止，得到的割一般不是最小割。

1.2 多源、多汇与指定流量

多源多汇：增加超级源 `S` 和超级汇 `T`。`S -> 每个源点`、`每个汇点 -> T` 的容量取该点允许的供给/接收量；若无限制则取足够大的 `INF`。

要求恰好发送 `k` 单位流：运行 `f.maxflow(s,t,k)`，返回值等于 `k` 才可行。仅需判断是否至少能发送 `k` 时也相同。

---

2. HLPP：高标号预流推进

当普通 Dinic 在大规模稠密图、层次很差的图上偏慢时使用。模板采用最高标号选择和 gap 优化，实践中通常很快；通用最坏复杂度为 `O(n^3)`。

- `HLPP f(n)`：建立 `1..n`。
- `add(u,v,c,rc)`：加入有向边；无向容量边令 `rc=c`。
- `maxflow(s,t)`：求最大流。该实现是**一次性**的，调用后容量已改变；需要重跑时重新建图。

```
struct HLPP {
    struct E { int v, r; ll f, c; };
    int n;
    vector<vector<E>> g;
    vector<ll> ex;
    vector<int> h, cnt, cur;
    vector<vector<int>> b;

    HLPP(int n) : n(n), g(n + 1) {}
    void add(int u, int v, ll c, ll rc = 0) { // u->v容量c; rc: 初始反向容量
        if (u == v) return;
        E a{v, (int)g[v].size(), 0, c};
        E z{u, (int)g[u].size(), 0, rc};
        g[u].push_back(a), g[v].push_back(z);
    }
    void push(int u, E &x, ll z) {
        E &y = g[x.v][x.r];
        if (ex[x.v] == 0 && z) b[h[x.v]].push_back(x.v);
        x.f += z, x.c -= z, ex[x.v] += z;
        y.f -= z, y.c += z, ex[u] -= z;
    }
    ll maxflow(int s, int t) { // s,t: 源汇
        if (s == t) return 0;
        int N = n + 1, H = 2 * N + 2;
        ex.assign(N, 0), h.assign(N, 0), cur.assign(N, 0);
        cnt.assign(H, 0), b.assign(H, {});
        h[s] = N, ex[t] = 1; // ex[t]的1是防止汇点入队的哨兵
        cnt[0] = N - 1, cnt[N] = 1;
        for (auto &x : g[s]) if (x.c) push(s, x, x.c);

        int hi = 0;
        while (1) {
            while (hi >= 0 && b[hi].empty()) --hi;
            if (hi < 0) break;
            int u = b[hi].back(); b[hi].pop_back();
            while (ex[u] > 0) {
                if (cur[u] == (int)g[u].size()) {
```

```
                    int old = h[u], nh = H - 1, at = 0;
                    for (int i = 0; i < (int)g[u].size(); i++) {
                        auto &x = g[u][i];
                        if (x.c > 0 && nh > h[x.v] + 1)
                            nh = h[x.v] + 1, at = i;
                    }
                    h[u] = nh, cur[u] = at;
                    --cnt[old], ++cnt[h[u]];
                    if (cnt[old] == 0 && old < N) {
                        for (int v = 0; v < N; v++) {
                            if (old < h[v] && h[v] < N) {
                                --cnt[h[v]], h[v] = N + 1, ++cnt[h[v]];
                            }
                        }
                    }
                    hi = h[u];
                } else {
                    E &x = g[u][cur[u]];
                    if (x.c > 0 && h[u] == h[x.v] + 1)
                        push(u, x, min(ex[u], x.c));
                    else
                        ++cur[u];
                }
            }
        }
        return -ex[s];
    }
};
```

HLPP 的邻接表中 `c` 是**当前残量**，`f` 是净流量。若需要稳定提取最小割、重复复用图或处理上下界，优先使用上一节 Dinic。

---

3. 最小割与常见最大流建模

建模的核心是：要让一次割的容量准确表示一种方案的代价，并用 `INF` 表示不可违反的依赖。`INF` 应严格大于所有有限容量之和；直接用公共常量的前提是答案不会溢出。

3.1 最大权闭合子图 / 项目选择

每个点有收益 `w[i]`，选一个点后必须选择它依赖的所有点，求最大总收益。

- `w[i] > 0`：连 `S -> i`，容量 `w[i]`。
- `w[i] < 0`：连 `i -> T`，容量 `-w[i]`。
- 若选 `u` 必须选 `v`：连 `u -> v`，容量 `INF`。
- 答案为 `所有正收益之和 - maxflow(S,T)`。
- 最大流后 `cut(S)` 中仍从 `S` 可达的原点，就是一组最优选择。

```
ll sum = 0;
for (int i = 1; i <= n; i++) {
    if (w[i] > 0) f.add(S, i, w[i]), sum += w[i];
    if (w[i] < 0) f.add(i, T, -w[i]);
}
for (auto [u, v] : dep) f.add(u, v, INF); // u依赖v
ll ans = sum - f.maxflow(S, T);
```

“项目需要若干材料”“选课程有先修课”“获利点与付费点”等都属于该模型。

3.2 二分图最大匹配与最小点覆盖

左部点连 `S -> u` 容量 1，右部点连 `v -> T` 容量 1，每条二分图边连 `u -> v` 容量 1。最大流就是最大匹配。

最大流结束后令 `vis=f.cut(S)`，一组最小点覆盖为：

- 左部中 `!vis[u]` 的点；
- 右部中 `vis[v]` 的点。

点覆盖大小等于最大匹配大小。纯二分图匹配且数据很大时，Hopcroft-Karp 的常数和复杂度通常更优。

3.3 最小路径覆盖（DAG）

把每个点拆成左、右两个副本；原 DAG 中每条边 `u -> v`，在二分图中加入 `u左 -> v右`。若要求用最少条点不相交路径覆盖全部点：

`最小路径条数 = n - 二分图最大匹配数`。

这条结论要求原图是 DAG。用匹配边可还原每条路径。

3.4 点容量、点割与不相交路径

把原点 `u` 拆为 `uin -> uout`，容量为该点允许通过的次数或删除代价。所有原边改成从 `uout` 指向 `vin`。

- 求最小点割：拆点边容量为删除点的代价，其他边容量为 `INF`。
- 求内部点不相交路径条数：普通点拆点容量为 1，源汇容量为 `INF`。
- 求边不相交路径条数：原边容量为 1，无需拆点。

若源点或汇点不能被删除，它们的拆点边容量应设为 `INF`。

3.5 二选一与冲突关系

最小割适合表达二元状态：点在 `S` 侧表示选择状态 0，在 `T` 侧表示状态 1。

- 单点选择 0 的代价为 `a`、选择 1 的代价为 `b`：加 `S -> u` 容量 `b`，`u -> T` 容量 `a`。
- 两点状态不同时付代价 `c`：无向容量边 `add(u,v,c,c)`。

更一般的二元能量需要满足可割性（次模条件），不能把任意四种组合代价直接塞进最小割。

3.6 最小割唯一性与必割边

跑满最大流后：

- `A=f.cut(S)`：残量网络中从 `S` 可达，属于**每一个**最小割的源侧。
- `B=f.rcut(T)`：残量网络中能到达 `T`，属于**每一个**最小割的汇侧。
- 对每个点都有 `A[i] || B[i]`，当且仅当最小割的点集划分唯一。
- 一条正容量原边 `u -> v` 出现在**每一个**最小割中，当且仅当 `A[u] && B[v]`。

```
f.maxflow(S, T);
auto A = f.cut(S), B = f.rcut(T);
bool one = true;
for (int i = 1; i <= n; i++) one &= A[i] || B[i]; // one: 最小割划分是否唯一
for (auto [u, v, id] : a) {
    bool must = f.cap(id) > 0 && A[u] && B[v]; // must: 该原边是否必在所有最小割
}
```

`A` 给出源侧最小的最小割，`V-B` 给出源侧最大的最小割。中间点如何分侧还必须满足残量边的闭合关系，不能任意选择。

需要描述全部最小割时，可把残量网络缩成 SCC DAG。每个最小割的源侧恰好对应一个包含 `S` 所在 SCC、不包含 `T` 所在 SCC、并且对残量出边闭合的 SCC 集合；这是计数或枚举全部最小割的标准入口。

---

4. 有上下界的流

每条边要求 `l <= flow(u,v) <= r`。先固定流过下界 `l`，把原边容量改为 `r-l`，再用超级源 `SS`、超级汇 `TT` 补齐各点流量平衡。

- `LB f(n,m)`：建立有上下界网络。
- `add(u,v,l,r)`：加入下界 `l`、上界 `r` 的有向边，返回该原边编号。
- `supply(u,x)`：令点 `u` 必须净流出 `x`；需求点可写成 `supply(u,-x)`。
- `circulation()`：判断是否存在可行循环流。
- `max_st(s,t)`：求可行的 `s-t` 最大流，返回 `{是否可行,答案}`。
- `min_st(s,t)`：求可行的 `s-t` 最小流，返回 `{是否可行,答案}`。
- `get(k)`：求第 `k` 条原边最终流量。

同一个对象只能调用三种求解函数中的一种；求另一种答案时重新建图。

```
struct LB {
    int n, ss, tt;
    ll need = 0;
    Dinic f;
    vector<ll> d, lo;
    vector<int> id, aux;

    LB(int n, int m = 0) : n(n), ss(n + 1), tt(n + 2),
```

```
        f(n + 2, m + n + 1), d(n + 1) {}

    int add(int u, int v, ll l, ll r) { // u->v; l,r: 流量上下界
        int z = f.add(u, v, r - l);
        d[u] -= l, d[v] += l;
        id.push_back(z), lo.push_back(l);
        return (int)id.size() - 1;
    }
    void supply(int u, ll x) { d[u] += x; } // u: 点; x: 净供给量(需求取负)
    bool build() {
        ll take = 0;
        for (int i = 1; i <= n; i++) {
            if (d[i] > 0) aux.push_back(f.add(ss, i, d[i])), need += d[i];
            if (d[i] < 0) aux.push_back(f.add(i, tt, -d[i])), take -= d[i];
        }
        return need == take && f.maxflow(ss, tt) == need;
    }
    void clear_aux() { for (int z : aux) f.off(z); }

    bool circulation() { return build(); }

    pair<bool, ll> max_st(int s, int t) { // s,t: 原网络源汇
        int z = f.add(t, s, INF);
        if (!build()) return {false, 0};
        ll base = f.flow(z);
        clear_aux(), f.off(z);
        return {true, base + f.maxflow(s, t)};
    }
    pair<bool, ll> min_st(int s, int t) { // s,t: 原网络源汇
        int z = f.add(t, s, INF);
        if (!build()) return {false, 0};
        ll base = f.flow(z);
        clear_aux(), f.off(z);
        return {true, base - f.maxflow(t, s)};
    }
    ll get(int k) const { return lo[k] + f.flow(id[k]); } // k: add返回编号
};
```

可行循环流要求每个点流入等于流出。`build()` 中所有从 `ss` 发出的边都满流，当且仅当存在可行方案。

若题目直接给每个点的供给/需求，供给 `x` 调用 `supply(u,x)`，需求 `x` 调用 `supply(u,-x)`；所有供给与需求之和必须为 0。此时使用 `circulation()` 求解，原边流量仍由 `get(k)` 查询。

```
LB f(n, m);
for (int i = 0; i < m; i++) {
    cin >> u >> v >> l >> r;
    f.add(u, v, l, r); // u,v: 端点; l,r: 下界和上界
}
if (!f.circulation()) cout << "NO\n";
else {
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << f.get(i) << '\n';
}
```

如果题意要求所有边流量非负且 `l,r` 合法，调用前必须保证 `0 <= l <= r`。上下界总和和最终答案必须小于 `INF`。

---

5. 最小费用流：势能 Dijkstra

每条边除容量外还有单位费用 `w`，发送 `x` 单位流的费用为 `x*w`。模板先用 SPFA 求一次初始势能，因此允许原图出现负费用边；之后每轮在非负约化费用上跑 Dijkstra。

- `MCMF f(n,m)`：建立 `1..n`，`m` 为原边数预估。
- `add(u,v,c,w)`：加入容量 `c`、单位费用 `w` 的有向边，返回边编号。
- `flow(s,t,lim)`：最多发送 `lim` 流，返回 `{实际流量,最小费用}`。省略 `lim` 即求最小费用最大流。
- `slope(s,t,lim)`：返回流量-最小费用折线的拐点，包含 `{0,0}`；相邻相同边际费用会合并。
- `used(id)`：查询 `add` 返回的原边已使用流量。
- 若要求恰好发送 `k`，调用 `flow(s,t,k)` 并检查返回流量是否等于 `k`。

假设从源点可达的残量网络中不存在负费用环。若存在可利用的负环，单纯沿 `s-t` 最短路增广不能完成一般的最小费用循环流问题。

```
struct MCMF {
    struct E { int v, nx; ll c, w; };
    int n;
    vector<int> h, pre;
    vector<E> e;
    vector<ll> p, d;

    MCMF(int n = 0, int m = 0) { init(n, m); }
    void init(int n_, int m = 0) { // n_: 点数; m: 原边数预估
        n = n_;
        h.assign(n + 1, -1), pre.resize(n + 1);
        p.resize(n + 1), d.resize(n + 1);
        e.clear(), e.reserve(2 * m);
    }
    int add(int u, int v, ll c, ll w) { // u->v; c: 容量; w: 单位费用
        int id = (int)e.size();
        e.push_back({v, h[u], c, w}), h[u] = id;
```

```
        e.push_back({u, h[v], 0, -w}), h[v] = id ^ 1;
        return id;
    }
    void init_p(int s) { // s: 源点; 处理初始负费用
        fill(p.begin(), p.end(), INF);
        vector<char> in(n + 1);
        deque<int> q;
        p[s] = 0, q.push_back(s), in[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop_front(), in[u] = 0;
            for (int i = h[u]; i != -1; i = e[i].nx) if (e[i].c > 0) {
                int v = e[i].v;
                if (p[v] > p[u] + e[i].w) {
                    p[v] = p[u] + e[i].w;
                    if (!in[v]) {
                        in[v] = 1;
                        if (!q.empty() && p[v] < p[q.front()]) q.push_front(v);
                        else q.push_back(v);
                    }
                }
            }
        }
        for (int i = 1; i <= n; i++) if (p[i] == INF) p[i] = 0;
    }
    bool dij(int s, int t) {
        fill(d.begin(), d.end(), INF);
        fill(pre.begin(), pre.end(), -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                       greater<pair<ll, int>>> q;
        d[s] = 0, q.push({0, s});
        while (!q.empty()) {
            auto [du, u] = q.top(); q.pop();
            if (du != d[u]) continue;
            for (int i = h[u]; i != -1; i = e[i].nx) if (e[i].c > 0) {
                int v = e[i].v;
                ll nd = du + e[i].w + p[u] - p[v];
                if (d[v] > nd) d[v] = nd, pre[v] = i, q.push({nd, v});
            }
        }
        if (d[t] == INF) return false;
        for (int i = 1; i <= n; i++) if (d[i] < INF) p[i] += d[i];
        return true;
    }
    vector<pair<ll, ll>> slope(int s, int t, ll lim = INF) { // s,t: 源汇; lim: 流量上
限
        vector<pair<ll, ll>> ans{{0, 0}};
        if (s == t || lim <= 0) return ans;
        ll fl = 0, co = 0;
        ll last = INF;
        init_p(s);
        while (fl < lim && dij(s, t)) {
            ll z = lim - fl, pc = 0;
```

```
            for (int v = t; v != s; v = e[pre[v] ^ 1].v) {
                z = min(z, e[pre[v]].c);
                pc += e[pre[v]].w;
            }
            for (int v = t; v != s; v = e[pre[v] ^ 1].v) {
                int i = pre[v];
                e[i].c -= z, e[i ^ 1].c += z;
                co += z * e[i].w;
            }
            fl += z;
            if (pc == last) ans.back() = {fl, co};
            else ans.push_back({fl, co}), last = pc;
        }
        return ans;
    }
    pair<ll, ll> flow(int s, int t, ll lim = INF) { // s,t: 源汇; lim: 流量上限
        return slope(s, t, lim).back();
    }
    ll used(int id) const { return e[id ^ 1].c; } // id: add返回值
};
```

设实际增广次数为 `A`，复杂度约为一次 SPFA 加 `O(A m log n)`。容量很大时一次可推送整条路径的瓶颈，不是逐单位增广；但最坏情况下 `A` 仍可能很大。

5.1 常用变形

- 最大费用最大流：把所有费用取相反数，求得最小费用后再把答案费用取反。
- 恰好 `k` 流量的最小费用：`auto [fl,co]=f.flow(s,t,k)`，只有 `fl==k` 才可行。
- 不强制流量最大，只选有利方案：调用 `slope` 后在折线拐点中选择费用最小者；若要对应方案，用选出的流量上限重新建图运行 `flow`。
- 多次查询不同流量的最小费用：先求 `slope`；同一线段内边际费用不变，可由两端点线性计算，无需重复跑费用流。
- 费用可能为小数时不要直接使用浮点；通常乘统一倍数转成整数。

5.2 带权二分图匹配 / 指派

`S -> 左部`、`右部 -> T` 容量均为 1；候选匹配边 `u -> v` 容量 1、费用为匹配代价。发送 `k` 流就是选 `k` 对且总代价最小。

若要求最大权匹配，把权值取负作为费用。若必须完美匹配，令 `k` 等于较小一侧点数并检查能否送满。

5.3 带上下界的最小费用可行流

同时给出每条边的下界、上界和单位费用，求满足流量平衡的最小费用方案。模板支持正负费用：

- 非负费用边先固定在下界，只保留“增加流量”的非负费用残量边。
- 负费用边先固定在上界，只保留“减少流量”的反向非负费用残量边。
- 再从超级源到超级汇跑一次最小费用定流量。

这样初始残量边费用均非负，同时不会漏掉负费用环带来的收益。

- `MCirc f(n,m)`：建立 `1..n` 的带费用循环流。
- `add(u,v,l,r,w)`：加入流量范围 `[l,r]`、单位费用 `w` 的有向边，返回原边编号。
- `supply(u,x)`：点 `u` 净供给 `x`；需求 `x` 写成负数。
- `solve()`：返回 `{是否可行,最小费用}`，只能调用一次。
- `get(k)`：求第 `k` 条原边在最优方案中的实际流量。

```
struct MCirc {
    struct R { int id, sg; ll base; };
    int n, ss, tt;
    ll bc = 0;
    MCMF f;
    vector<ll> d;
    vector<R> a;

    MCirc(int n, int m = 0) : n(n), ss(n + 1), tt(n + 2),
        f(n + 2, m + n), d(n + 1) {}

    int add(int u, int v, ll l, ll r, ll w) { // u->v; l,r: 上下界; w: 费用
        ll x;
        int id, sg;
        if (w >= 0) {
            x = l, id = f.add(u, v, r - l, w), sg = 1;
        } else {
            x = r, id = f.add(v, u, r - l, -w), sg = -1;
        }
        d[u] -= x, d[v] += x, bc += x * w;
        a.push_back({id, sg, x});
        return (int)a.size() - 1;
    }
    void supply(int u, ll x) { d[u] += x; } // u: 点; x: 净供给量(需求取负)

    pair<bool, ll> solve() {
        ll pos = 0, neg = 0;
        for (int i = 1; i <= n; i++) {
            if (d[i] > 0) f.add(ss, i, d[i], 0), pos += d[i];
            if (d[i] < 0) f.add(i, tt, -d[i], 0), neg -= d[i];
        }
        if (pos != neg) return {false, 0};
        auto [fl, co] = f.flow(ss, tt, pos);
        return fl == pos ? make_pair(true, bc + co) : make_pair(false, 0LL);
    }
    ll get(int k) const { // k: add返回编号
        return a[k].base + a[k].sg * f.used(a[k].id);
    }
};
```

恰好发送 `k` 单位 `s-t` 流时，额外加入固定流量边 `add(t,s,k,k,0)`，再求最小费用循环流。若流量只要求在 `[L,R]`，加入 `add(t,s,L,R,0)`。

要求"带上下界的最小费用最大流"时，先用 `LB::max_st` 求最大流量 `F`；重新建立 `MCirc`，加入固定边 `t -> s`、上下界 `[F,F]`，即可在所有最大流方案中取最小费用。

---

6. Gomory-Hu Tree：无向图任意两点最小割

对带非负容量的无向图构造一棵割树。树上任意两点路径的最小边权，等于原图中这两点之间的最小割。构树需要 `n-1` 次最大流；完成倍增后单次查询 `O(log n)`。

- `GH g(n,m)`：建立无向图，`m` 为无向边数预估。
- `add(u,v,c)`：加入容量为 `c` 的无向边。
- `build()`：构造割树和倍增表，只调用一次。
- `ask(u,v)`：查询不同点 `u,v` 之间的最小割。
- `tr`：最终割树邻接表，可用于离线处理全部询问。

```
struct GH {
    int n, lg;
    Dinic f;
    vector<int> p, dep;
    vector<ll> w;
    vector<vector<pair<int, ll>>> tr;
    vector<vector<int>> up;
    vector<vector<ll>> mn;

    GH(int n, int m = 0) : n(n), f(n, m), tr(n + 1) {}
    void add(int u, int v, ll c) { // u-v: 无向边; c: 容量
        f.add(u, v, c, c);
    }
    void build() {
        p.assign(n + 1, 1), w.assign(n + 1, 0);
        p[1] = 0;
        for (int s = 2; s <= n; s++) {
            int t = p[s];
            f.reset();
            ll z = f.maxflow(s, t);
            auto vis = f.cut(s);
            for (int i = s + 1; i <= n; i++)
                if (p[i] == t && vis[i]) p[i] = s;
            if (vis[p[t]]) {
                p[s] = p[t], p[t] = s;
                w[s] = w[t], w[t] = z;
            } else {
                w[s] = z;
            }
        }
        tr.assign(n + 1, {});
```

```
        for (int i = 2; i <= n; i++) {
            tr[i].push_back({p[i], w[i]});
            tr[p[i]].push_back({i, w[i]});
        }

        lg = 1;
        while ((1 << lg) <= n) ++lg;
        up.assign(lg, vector<int>(n + 1));
        mn.assign(lg, vector<ll>(n + 1, INF));
        dep.assign(n + 1, -1);
        queue<int> q;
        dep[1] = 0, q.push(1);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto [v, c] : tr[u]) if (v != up[0][u]) {
                dep[v] = dep[u] + 1;
                up[0][v] = u, mn[0][v] = c;
                q.push(v);
            }
        }
        for (int k = 1; k < lg; k++) {
            for (int v = 1; v <= n; v++) {
                up[k][v] = up[k - 1][up[k - 1][v]];
                mn[k][v] = min(mn[k - 1][v], mn[k - 1][up[k - 1][v]]);
            }
        }
    }
    ll ask(int u, int v) const { // u,v: 两个不同点
        ll ans = INF;
        if (dep[u] < dep[v]) swap(u, v);
        int d = dep[u] - dep[v];
        for (int k = 0; k < lg; k++) if (d >> k & 1)
            ans = min(ans, mn[k][u]), u = up[k][u];
        if (u == v) return ans;
        for (int k = lg - 1; k >= 0; k--) if (up[k][u] != up[k][v]) {
            ans = min({ans, mn[k][u], mn[k][v]});
            u = up[k][u], v = up[k][v];
        }
        return min({ans, mn[0][u], mn[0][v]});
    }
};
```

平行边可以直接加入，会自然累加割容量；自环对割无贡献，可忽略。图不连通时也可构造，跨连通块的查询答案为 0。`ask(u,u)` 没有实际意义，本模板要求两点不同。

如果只需无向图的全局最小割，而不是任意点对询问，直接使用 Stoer-Wagner 通常更省代码和时间。

---

7. 全局最小割

7.1 Stoer-Wagner：无向图全局最小割

不指定源汇，求无向非负容量图中最小的割。支持重边，复杂度 `O(n^3)`、空间 `O(n^2)`，适合点数较小或较稠密的图。

- 输入矩阵 `w` 使用 `1..n`，大小为 `(n+1)*(n+1)`。
- 无向边 `u-v` 容量为 `c`：执行 `w[u][v]+=c, w[v][u]+=c`。
- 返回 `{最小割容量,割的一侧原点集合}`。
- 原图不连通时答案为 0；自环直接忽略。

```
pair<ll, vector<int>> stoer(vector<vector<ll>> w) { // w: 1-index 无向容量矩阵
    int n = (int)w.size() - 1;
    if (n < 2) return {0, {}};
    vector<int> v(n);
    iota(v.begin(), v.end(), 1);
    vector<vector<int>> g(n + 1);
    for (int i = 1; i <= n; i++) g[i] = {i};
    ll ans = INF;
    vector<int> best;

    while (v.size() > 1) {
        vector<ll> d(n + 1);
        vector<char> vis(n + 1);
        int pre = 0, sel = 0;
        for (int k = 0; k < (int)v.size(); k++) {
            sel = 0;
            for (int x : v) if (!vis[x] && (sel == 0 || d[x] > d[sel]))
                sel = x;
            if (k + 1 == (int)v.size()) {
                if (d[sel] < ans) ans = d[sel], best = g[sel];
                for (int x : v) if (x != sel) {
                    w[pre][x] += w[sel][x];
                    w[x][pre] = w[pre][x];
                }
                g[pre].insert(g[pre].end(), g[sel].begin(), g[sel].end());
                v.erase(find(v.begin(), v.end(), sel));
                break;
            }
            vis[sel] = 1, pre = sel;
            for (int x : v) if (!vis[x]) d[x] += w[sel][x];
        }
    }
    return {ans, best};
}
```

稀疏图且已有 Dinic 时，也可任选根 `r`，计算所有 `mincut(r,v)`，最小值就是无向全局最小割；构造完整 Gomory-Hu Tree 后取树上最小边权也相同。

7.2 有向图全局最小割

有向图中，割容量只统计从源侧指向汇侧的边。任选固定根 `r`：任意非平凡割要么把 `r` 留在源侧，要么留在汇侧，因此枚举 `maxflow(r,v)` 与 `maxflow(v,r)` 的最小值即可。

需要 `2(n-1)` 次最大流，适合 `n` 不太大但不想实现 Hao-Orlin 的场景。

```
ll global_dir(Dinic &f, int n, int r = 1) { // f: 原有向图; n: 点数; r: 固定根
    if (n < 2) return 0;
    ll ans = INF;
    for (int v = 1; v <= n; v++) if (v != r) {
        f.reset(), ans = min(ans, f.maxflow(r, v));
        f.reset(), ans = min(ans, f.maxflow(v, r));
    }
    return ans;
}
```

函数结束后 `f` 保留最后一次最大流的残量状态；继续做其他操作前调用 `reset()`。若要输出最优割点集，在刷新答案时保存当前 `f.cut(源点)`。

---

8. 进阶建模提示

8.1 最大密度子图 / 分数规划

形如最大化 `收益 / 代价` 的问题，常二分答案 `x`，转成判断是否存在 `收益 - x*代价 > 0` 的方案，再用最大权闭合子图或最小割判断。浮点二分需要留足精度；若数据均为整数且要求精确分数，可用分数规划或枚举候选比值。

8.2 时间展开网络

点状态与时间有关时，把 `(点,时刻)` 作为新点：

- 等待边 `(u,t) -> (u,t+1)`；
- 行动边 `(u,t) -> (v,t+耗时)`；
- 边容量表示该时段允许通过的数量。

这能处理分时运输、动态避碰、每轮容量限制等问题，但点边数会乘时间范围，应先估算内存。

8.3 流量方案还原

保存每条原边的 Dinic 编号 `id`，其净流量为 `f.flow(id)`。若要拆成若干条 `s-t` 路径，在所有正流量原边组成的图上反复 DFS 找路径，并减去路径瓶颈；剩余正流量可能是环流，可继续拆环或在题目不要求时忽略。

- `DE{u,v,id}` 保存一条原有向边及 Dinic 编号。
- `decomp(n,s,t,f,a)` 返回若干个 `{流量,原边下标序列}`，这些路径之和等于 `s-t` 流量。
- 每返回一条路径至少清空一条正流边，路径数不超过原边数，复杂度 `O(k(n+m))`。

```cpp
struct DE { int u, v, id; };
struct FP { ll f; vector<int> e; };

vector<FP> decomp(int n, int s, int t, const Dinic &f,
                  const vector<DE> &a) { // n,s,t: 图与源汇; a: 原有向边
    int m = (int)a.size();
    vector<ll> x(m);
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        x[i] = max(0LL, f.flow(a[i].id));
        if (x[i] > 0) g[a[i].u].push_back(i);
    }
    vector<FP> ans;
    while (1) {
        vector<int> pre(n + 1, -1);
        queue<int> q;
        pre[s] = -2, q.push(s);
        while (!q.empty() && pre[t] == -1) {
            int u = q.front(); q.pop();
            for (int j : g[u]) if (x[j] > 0 && pre[a[j].v] == -1) {
                pre[a[j].v] = j, q.push(a[j].v);
            }
        }
        if (pre[t] == -1) break;
        ll z = INF;
        vector<int> p;
        for (int v = t; v != s; v = a[pre[v]].u)
            z = min(z, x[pre[v]]), p.push_back(pre[v]);
        reverse(p.begin(), p.end());
        for (int j : p) x[j] -= z;
        ans.push_back({z, move(p)});
    }
    return ans;
}
```

模板只拆出 `s-t` 路径并忽略纯环流，适合输出运输路线、匹配链等。若题目还要求输出环，对剩余 `x[i]>0` 的边继续找有向环并减去环上瓶颈即可。

费用流模板中正向边编号同样是 `add` 的返回值，流量可用 `used(id)` 查询。

8.4 负容量与无穷边

网络流容量不能为负。建模式子出现负数时，通常需要把常数项移出、交换源汇方向或按正负收益分别连边。

`INF` 不是越大越好：取“所有有限容量之和 + 1”最严谨。若直接用 `(1LL<<62)`，确认任意累加、费用乘法和答案都不会溢出。

---

9. 赛场检查清单

1. 点编号是否与模板的 `1..n` 一致，超级源汇是否计入构造函数点数。

2. 有向边、无向容量边是否区分：有向 `add(u,v,c)`，无向 `add(u,v,c,c)`。
3. 容量、总流量、费用、`流量*费用` 是否都能装进 `ll`。
4. `INF` 是否严格大于任意合法有限割，同时不会参与溢出运算。
5. Dinic 的 `maxflow` 返回的是**本次新增流量**；连续调用时要自行累加。
6. 提取最小割前是否已经跑满最大流，而不是被 `lim` 提前截断。
7. 上下界流是否满足 `0<=l<=r`，并且一个 `LB` 对象只调用一种求解模式。
8. 费用流有负边时是否保留了 `init_p`；是否检查指定流量真的送满。
9. 最大费用是否已经对费用取负，并在输出时把费用答案符号改回。
10. Gomory-Hu 只适用于无向非负容量图；有向图不能套用割树。
11. 拆点后所有原边端点方向是否从 `uout` 指向 `vin`。
12. 输出方案时是否保存了 `add` 返回的正向边编号，避免把反向残量当原边。
13. 点供需的符号是否一致：本板子正数表示净供给，负数表示净需求，二者总和必须为 0。
14. Stoer-Wagner 的容量矩阵是否双向累加，且没有把有向图误当无向图。
15. `MCirc::solve()` 是否只调用一次；固定 `s-t` 流量时是否加入了方向为 `t -> s` 的固定流量边。